Proyecto intermodular I de DAM

None

Cristian Garcia

None

proyectointermodular

1. Bienvenid@ al módulo de Proyecto Intermodular 1 de Desarrollo de Aplicaciones Multiplataforma 3
   1.1 Bolsa de proyectos 3
2. Sprint 3. Cronograma y planificación temporal del proyecto 4
   2.1 Semana 7. Secuenciación de tareas, cronograma y Sprint Backlog 3 4
   2.2 Semana 9. Consolidación de la memoria técnica y demo del incremento v0.3 7
3. Sprint 2. Especificación de requisitos del sistema 13
   3.1 Semana 4. Catálogo de requisitos y Sprint Backlog 2 13
   3.2 Semana 5. Modelado visual y previsión de recursos 16
   3.3 Semana 6. Consolidación de la memoria y demo técnica v0.2 19

1. Bienvenid@ al módulo de Proyecto Intermodular 1 de Desarrollo de Aplicaciones Multiplataforma

El equipo de estudiantes debe elegir uno de los proyectos que se muestran a continuación.

1.1 Bolsa de proyectos

1.1.1 Ciberseguridad

1. Simulador de phishing — juego interactivo de detección de mensajes fraudulentos; clasificación de mensajes por tipología (Email, SMS, Red Social) con pistas contextuales, gestión de usuarios/sesiones, estadísticas de aciertos e historial de partidas.
2. Bóveda de contraseñas — gestor de credenciales con evaluación de fortaleza mediante políticas de seguridad (Básica, Estricta, Corporativa); control de caducidad, filtrado de cuentas guardadas y persistencia del almacén seguro.

1.1.2 Python

1. Calculadora paso a paso — evaluador de expresiones matemáticas con desglose paso a paso; tokenización, precedencia y cálculo con operadores polimórficos, interfaz con visor del proceso de resolución e histórico de cálculos.
2. Generador de datos de prueba — herramienta de síntesis de datasets sintéticos a partir de plantillas; campos polimórficos (Texto, Número, Fecha, Email), generación de lotes de registros aleatorios y exportación a ficheros CSV.

1.1.3 Videojuegos y realidad virtual

1. Aventura conversacional — motor de narrativa interactiva ramificada con toma de decisiones; gestión de escenas, inventario del jugador, interacción con diferentes tipos de personajes (Aliado, Enemigo, Neutral) y guardado del estado de la partida.
2. Simulador de físicas 2D — entorno de simulación cinemática con detección de colisiones y rebotes; escenario visual con múltiples bolas simultáneas de comportamiento polimórfico (Normal, Pesada, Rebote especial) y exportación de estadísticas de la simulación.

1.1.4 Productos software en contenedores

1. Reparto de contenedores en nodos — planificador de asignación de carga en clúster según capacidad de CPU y memoria; gestión de nodos especializados (Alta Memoria, Alta CPU), interfaz gráfica con barras de ocupación y persistencia de configuraciones de reparto.
2. Catálogo de servicios + generador de config — asistente paso a paso (wizard) para la definición y parametrización de servicios (Web, BD, Worker); generación polimórfica de ficheros de configuración, catálogo de servicios e historial de despliegues.

1.1.5 Inteligencia artificial y big data

1. Motor de recomendación — sistema de recomendación basado en afinidad entre gustos y catálogos de ítems categorizados (Película, Música, Libro); cálculo de puntuaciones de coincidencia por etiquetas, gestión de perfiles y persistencia de valoraciones.
2. Analizador de reseñas — analizador de sentimiento léxico sobre comentarios; procesamiento de palabras clave mediante diccionarios específicos según el tipo de producto, estadísticas agregadas por producto y persistencia del catálogo de reseñas.

1.1.6 Recursos y servicios en la nube

1. Cotizador cloud — presupuestador interactivo de infraestructura en la nube; cálculo polimórfico de costes según el tipo de recurso (Cómputo, Almacenamiento, Red), visor para montar y comparar presupuestos y persistencia de ofertas para clientes.
2. Dashboard de monitorización — panel de control visual para métricas y telemetría de sistemas; simulación de lecturas de sensores (CPU, Memoria, Red), series temporales, detección de umbrales con disparo de alertas e histórico persistido de incidencias.

2. Sprint 3. Cronograma y planificación temporal del proyecto

2.1 Semana 7. Secuenciación de tareas, cronograma y Sprint Backlog 3

2.1.1 Día 7 - 1 sesión

Teoría. Secuenciación de fases, estimación temporal y dependencias

1. Caso guía en AzaharTech

Es viernes 30 de octubre de 2026 por la tarde en la sede de **AzaharTech** en Castellón de la Plana. Tras una semana de intenso trabajo dominando las bifurcaciones y los bucles en Programación, junto con el manejo del depurador en Entornos de Desarrollo, el equipo se reúne en la sala de planificación para iniciar formalmente el **Sprint 3**.

En el panel interactivo, **Laia Claramunt** tiene desplegado un calendario que abarca desde la situación presente hasta el final del curso, junto con una estructura desglosada del sistema de control de acceso por QR del **IES El Caminàs**. A su lado, **Alba Torres** revisa la lista de tareas pendientes del motor de lectura y **Pau Ferrer** anota las duraciones estimadas de cada módulo.

Laia toma la palabra dirigiéndose al equipo:

«En los dos primeros sprints habéis sentado los cimientos: analizasteis las necesidades del cliente, redactasteis el catálogo de requisitos funcionales y no funcionales, y dibujasteis la arquitectura visual de la aplicación. Pero en la ingeniería del software real, una excelente arquitectura sin una planificación temporal rigurosa está condenada al fracaso.

*Si el equipo directivo del IES El Caminàs nos pregunta cuándo entrará en funcionamiento el lector del vestíbulo o qué ocurrirá si el módulo de validación se retrasa tres semanas, no podemos responder con suposiciones. Necesitamos una **Estructura de Descomposición del Trabajo (EDT)**, una matriz de duraciones y precedencias, y un **Diagrama de Gantt** que visibilice el camino crítico del proyecto.*

*Hoy aprenderemos a secuenciar las fases del desarrollo, estimaremos el esfuerzo en horas lectivas reales, construiremos el cronograma temporal combinando herramientas visuales y código declarativo, y abriremos el **Sprint Backlog 3**, que asegurará que el ritmo de trabajo sea constante, predecible y profesional».*

2. Fundamentos de secuenciación y planificación temporal

EL CICLO DE PLANIFICACIÓN TEMPORAL		
1. Descomposición (EDT)	2. Estimación y Dependencias	3. Cronograma (Gantt)
¿Qué paquetes de trabajo componen el producto de forma tangible?	¿Cuánto tiempo exige cada tarea y qué actividades deben finalizar antes?	¿En qué fechas exactas se ejecutan las fases y cuáles marcan el camino crítico?

A. ESTRUCTURA DE DESCOMPOSICIÓN DEL TRABAJO (EDT / WBS)

La Estructura de Descomposición del Trabajo (EDT, o *Work Breakdown Structure* en inglés) es una técnica fundamental de gestión que consiste en fragmentar un proyecto complejo en componentes más pequeños, manejables e independientes:

- **Orientación a entregables.** Cada elemento terminal de la EDT debe asociarse a un resultado verificable (un documento técnico formal, una clase Java compilada, un conjunto de casos de prueba o una etiqueta de versión en Git).
- **Nivel de granularidad adecuado.** Una tarea excesivamente genérica (*«programar la aplicación»*) genera incertidumbre; una tarea minúscula (*«declarar variable booleana»*) asfixia la gestión. El tamaño óptimo de un paquete de trabajo se sitúa entre las 2 y las 12 horas.
- **Criterio del 100 %:** La suma del trabajo de los niveles inferiores debe equivaler con total exactitud al alcance comprometido con el cliente, sin omitir tareas ni añadir requisitos fuera de contrato (*scope creep*).

B. ESTIMACIÓN DE TIEMPOS Y GESTIÓN DE DEPENDENCIAS ENTRE TAREAS

Ninguna tarea de desarrollo ocurre en el vacío. Para articular un cronograma creíble es imprescindible definir las relaciones lógicas entre actividades:

- **Dependencia Fin a Inicio (FS - *Finish to Start*).** Es la más habitual en software. Una actividad no puede comenzar hasta que su predecesora haya concluido satisfactoriamente (*ejemplo: no se puede iniciar la depuración de algoritmos en Entornos de Desarrollo sin tener implementada la lógica condicional en Programación*).
- **Hitos del proyecto (*Milestones*).** Puntos de control temporales de duración cero que marcan el cierre formal de una fase (*por ejemplo: H1 - Catálogo de requisitos cerrado, H2 - Entrega v0.3 del prototipo*).
- **Camino crítico.** La secuencia encadenada de tareas que determina la duración total mínima del proyecto. Cualquier retraso en una tarea situada en el camino crítico pospone de forma inmediata la fecha de entrega acordada con el cliente.

C. MODELADO DEL CRONOGRAMA: HERRAMIENTAS DE CÓDIGO Y SOFTWARE VISUAL

La representación de un cronograma debe ser accesible, mantenible y versionable junto con el resto del proyecto. En nuestro flujo de trabajo disponemos de dos alternativas complementarias:

1. **Modelado declarativo mediante código (Mermaid en Markdown).** Permite describir diagramas de Gantt mediante bloques de texto estructurado dentro de los propios documentos Markdown. Esta opción facilita el control de versiones en Git, la revisión de cambios mediante `git diff` y su visualización automática tanto en GitHub como en el visor interno del IDE.
2. **Modelado visual con software especializado de escritorio (GanttProject - https://www.ganttproject.biz/download/).** Para quienes prefieran diseñar y ajustar la planificación mediante una interfaz gráfica dedicada, **GanttProject** es una aplicación libre y multiplataforma orientada al cálculo de rutas críticas, dependencias y asignación de recursos. En entornos GNU/Linux puede ejecutarse descargando directamente su versión portable y lanzando el binario desde el directorio del usuario sin requerir instalaciones del sistema operativo, exportando el resultado en un archivo PNG.

D. FUENTES TÉCNICAS OFICIALES Y USO ÉTICO DE LA INTELIGENCIA ARTIFICIAL

- **Estándares internacionales de gestión.** La estructuración de las fases se apoya en las directrices de la norma ISO/IEC/IEEE 12207 (Procesos del ciclo de vida del software) y la Guía Scrum oficial.
- **Uso ético y crítico de la IA.** La Inteligencia Artificial resulta muy útil para obtener propuestas de desgloses de tareas iniciales y contrastar posibles dependencias técnicas que el analista novel suele pasar por alto. Sin embargo, los tiempos deben contrastarse siempre con el calendario lectivo oficial y las horas reales asignadas a cada módulo. Cada sugerencia de la IA debe documentarse de forma transparente en el *Prompt Log*.

Laboratorio. Planificación temporal, cronograma y Sprint Backlog 3

Laia Claramunt asigna los objetivos operativos para la sesión:

«Durante los próximos minutos, cada estudiante abrirá su repositorio y creará el documento formal de cronograma y secuenciación de su proyecto singular de la bolsa de proyectos. Modelaréis las fases del proyecto hasta el cierre del curso, definiréis la matriz de dependencias, incorporaréis el Diagrama de Gantt y crearéis el archivo ***`sprint3-backlog.md`*** *que coordinará las tres semanas que tenemos por delante».*

PASO 1. CREACIÓN DEL DOCUMENTO DE CRONOGRAMA (`pi/docs/cronograma.md`)

1. Abre tu proyecto en **IntelliJ IDEA**.
2. En la carpeta `azahartech/nombre-equipo/apellidos-nombre/pi/docs/` , crea el archivo `cronograma.md` .
3. Redacta el contenido aplicando la plantilla técnica adaptada a **tu proyecto de la bolsa de proyectos**:

```
# Planificación temporal, secuenciación de fases y cronograma
**Consultora:** AzaharTech Software Consulting
**Proyecto Seleccionado:** [nombre de tu proyecto de la bolsa de proyectos]
**Cliente / Sector:** [entidad receptora o sector profesional]
**Desarrollador/a:** [tus apellidos, tu nombre]
**Fecha:** 30 de octubre de 2026
**Versión:** 1.0 (Sprint 3)

---

## 1. Estructura de Descomposición del Trabajo (EDT)
El ciclo de vida del proyecto se divide en cuatro grandes fases secuenciales orientadas a entregables verificables:

* **Fase 1: Análisis y Viabilidad (Sprints 1 y 2 - Semanas 1 a 6).** Definición del problema, requisitos funcionales/no funcionales y arquitectura base.
* **Fase 2: Construcción del Núcleo y Control de Calidad (Sprints 3 y 4 - Semanas 7 a 12).** Implementación de la lógica algorítmica, validación de entradas, control de excepciones y auditoría de pruebas en consola.
* **Fase 3: Almacenamiento, Concurrencia e Interfaz Gráfica (Sprints 5 a 8 - Semanas 13 a 24).** Estructuras dinámicas de datos, persistencia en archivos estructurados y desarrollo de interfaces con Swing.
* **Fase 4: Consolidación Final y Pase a Producción (Semanas 25 a 30).** Pruebas de integración, empaquetado con Maven y defensa técnica pública.

---

## 2. Matriz de tareas, estimaciones y dependencias

| ID Tarea | Descripción del Paquete de Trabajo | Módulo | Duración Estimada | Dependencia (Predecesora) | Hito Asociado |
| :---: | :--- | :---: | :---: | :---: | :---: |
| **T-01** | Catálogo de requisitos y arquitectura IPO | PI | 6 h | Ninguna | H1: Base aprobada |
| **T-02** | Configuración de entorno Maven y Git | ED | 9 h | Ninguna | H1: Base aprobada |
| **T-03** | Lógica con clases estándar Java | PR | 48 h | Ninguna | Incremento v0.2 |
| **T-04** | Cronograma de fases y matriz de riesgos | PI | 3 h | T-01 | H2: Planificación |
| **T-05** | Implementación de bucles y control condicional | PR | 24 h | T-03 | Incremento v0.3 |
| **T-06** | Pruebas de código y depuración con breakpoints | ED | 9 h | T-05 | Incremento v0.3 |
| **T-07** | POO modular y encapsulamiento | PR | 24 h | T-05 | Prototipo v0.4 |
| **T-08** | Versionado de ramas y empaquetado | ED | 9 h | T-06 | Prototipo v0.4 |
| **T-09** | Dossier técnico y defensa final Sprint 4 | PI | 3 h | T-04, T-07 | H3: Fin Trimestre 1 |

---

## 3. Cronograma del proyecto (Diagrama de Gantt)

Elige una de las dos modalidades de representación técnica e incorpórala a este apartado:

### Opción A: Diagrama declarativo en código Mermaid

gantt
    title Cronograma de Desarrollo — AzaharTech Consulting
    dateFormat  YYYY-MM-DD
    axisFormat  %d/%m

    section Fase 1: Fundamentos y Requisitos
    Análisis y requisitos (Sprints 1 y 2)    :done, f1, 2026-09-14, 2026-10-23
    Hito 1 - Incremento v0.2 alcanzado       :milestone, m1, 2026-10-23, 0d

    section Fase 2: Lógica y Depuración
    Planificación temporal y riesgos (PI)    :active, t_pi3, 2026-10-26, 2026-11-13
    Estructuras de control y bucles (PR)     :active, t_pr3, 2026-10-26, 2026-11-13
    Depuración paso a paso e incidencias (ED):active, t_ed3, 2026-10-26, 2026-11-13
    Hito 2 - Incremento v0.3 consolidado     :milestone, m2, 2026-11-13, 0d

    section Fase 3: Arquitectura de Clases
    Diseño de clases y miembros (PR)        :t_pr4, 2026-11-16, 2026-12-04
    Ramas en Git y documentación (ED)        :t_ed4, 2026-11-16, 2026-12-04
    Dossier técnico y cierre T1 (PI)         :t_pi4, 2026-11-16, 2026-12-04
    Hito 3 - Prototipo individual completo   :milestone, m3, 2026-12-04, 0d

### Opción B: Diagrama exportado desde herramienta de escritorio

Si has modelado tu planificación en una herramienta de escritorio como **GanttProject**, guarda la imagen recortada en `pi/docs/img/cronograma-gantt.png` y referénciala aquí:

![Cronograma de Fases del Proyecto](img/cronograma.png)
*Figura 1. Cronograma temporal detallado y dependencias del sistema.*

---

## 4. Fuentes técnicas y estándares de gestión
* **ISO/IEC/IEEE 12207:2017:** Systems and software engineering — Software life cycle processes.
* **Scrum Guide (Ken Schwaber & Jeff Sutherland):** Principios de inspección, adaptación y definición del trabajo en sprints.
* **Documentación oficial de Oracle Java SE 21:** Estructuras sintácticas de bucles y gestión de excepciones en el lenguaje Java.

---

## 5. Registro ético de uso de Inteligencia Artificial (Prompt Log)
```

```
| Fecha | Herramienta | Objetivo de la consulta | Prompt introducido | Revisión crítica y ajuste aplicado |
| :---: | :---: | :--- | :--- | :--- |
| 30/10/2026 | ChatGPT / Claude | Estructurar dependencias del proyecto | *"Actúa como jefe de proyectos y sugiere una secuencia de dependencias lógicas para una aplicación Java que requiere validación de datos en consola..."* | Se ajustaron las duraciones al calendario de 3 semanas por sprint y se eliminaron tareas de base de datos no aplicables a esta fase. |
```

PASO 2. CREACIÓN DEL SPRINT BACKLOG 3 (`pi/backlog/sprint3-backlog.md`)

1. En la carpeta `azahartech/nombre-equipo/apellidos-nombre/pi/backlog/`, crea el archivo `sprint3-backlog.md`.
2. Registra los compromisos de trabajo del **Sprint 3 (26 oct – 13 nov)** coordinando los tres módulos:

```
# Sprint Backlog 3 — [nombre de tu proyecto]
**Periodo:** 26 de octubre – 13 de noviembre de 2026
**Responsable:** [tu nombre]

## Objetivo general del Sprint 3
Construir el incremento v0.3: definir la secuenciación temporal y matriz de riesgos en Proyecto Intermodular, dominar el uso de puntos de ruptura y seguimiento de variables en Entornos de Desarrollo, e implementar estructuras de control condicionales y repetitivas robustas en Programación.

---

## Tareas

### Módulo: Proyecto Intermodular (PI)
- [ ] T-PI-10: Elaborar la EDT, matriz de dependencias y Diagrama de Gantt (`pi/docs/cronograma.md` o `pi/docs/img/cronograma.png`)
- [ ] T-PI-11: Registrar el uso ético de IA (*Prompt Log*) en la planificación temporal
- [ ] T-PI-12: Redactar la matriz de riesgos técnicos y planes de contingencia (Semana 8)
- [ ] T-PI-13: Consolidar el Capítulo 3 de la memoria y ensayar la demo técnica v0.3 (Semana 9)

### Módulo: Entornos de Desarrollo (ED)
- [ ] T-ED-08: Diseñar casos de prueba de caja negra aplicando particiones de equivalencia y valores límite
- [ ] T-ED-09: Configurar puntos de ruptura (*breakpoints*) condicionales en IntelliJ IDEA para inspección de memoria
- [ ] T-ED-10: Documentar el procedimiento formal de recogida y registro de incidencias técnicas (Semana 8)

### Módulo: Programación (PR)
- [ ] T-PR-07: Implementar lógica condicional mediante bloques `if / else if / else` y selector `switch`
- [ ] T-PR-08: Construir bucles iterativos controlados (`while`, `for`) para la captura repetitiva de datos
- [ ] T-PR-09: Incorporar gestión de excepciones básica con bloques `try-catch` frente a entradas erróneas del usuario
```

PASO 3. CONFIRMACIÓN Y SINCRONIZACIÓN EN GITHUB

1. En IntelliJ IDEA, abre el panel de control de versiones **Commit** mediante el atajo de teclado (`Ctrl + K`).
2. Verifica que se encuentran seleccionados los archivos creados durante la sesión:

- *(Opcion A)* `pi/docs/cronograma.md` si has utilizado la opción de modelado declarativo.
- `pi/backlog/sprint3-backlog.md`
- *(Opcion B)* `pi/docs/img/cronograma.png` si has utilizado la opción modelado visual.

3. Escribe el mensaje de confirmación siguiendo el formato estandarizado de **Conventional Commits**:

```
docs(pi): definir secuenciacion de tareas, cronograma y abrir sprint backlog 3
```

1. Pulsa sobre el botón **Commit and Push**.
2. Accede a tu repositorio remoto en GitHub desde el navegador y verifica que los archivos aparecen estructurados correctamente y que el diagrama se visualiza con absoluta nitidez.

2.2 Semana 9. Consolidación de la memoria técnica y demo del incremento v0.3

2.2.1 Día 9 - 1 sesión

Teoría. Cierre del incremento de producto, consolidación documental y defensa técnica

1. Caso guía en AzaharTech

Es viernes 13 de noviembre de 2026 por la tarde en la sede de **AzaharTech** en Castellón de la Plana. Tras tres semanas de trabajo continuado, el equipo alcanza el cierre del **Sprint 3**.

En el proyector de la sala de reuniones, **Alba Torres** ejecuta la versión actualizada del lector QR para el **IES El Caminàs**. Introduce deliberadamente datos incorrectos, códigos en blanco y entradas no numéricas en la consola: el sistema responde con mensajes claros de advertencia, solicita de nuevo los datos mediante un bucle controlado y jamás interrumpe su ejecución. En otra pestaña, **Pau Ferrer** muestra el historial de incidencias resueltas en Entornos de Desarrollo tras localizar la causa raíz en memoria con el depurador de IntelliJ IDEA.

Laia Claramunt asiente satisfecha y toma el control de la sesión:

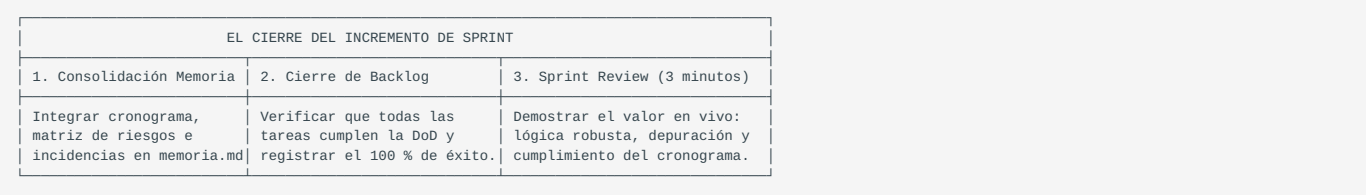
«Habéis logrado transformar un código frágil en una solución robusta. En Programación vuestro algoritmo valida y controla cualquier entrada imprevista; en Entornos de Desarrollo habéis verificado con breakpoints que el flujo en memoria es el adecuado y habéis registrado formalmente las incidencias técnicas; y en Proyecto Intermodular tenéis el proyecto secuenciado en el cronograma y los riesgos bajo control con planes de contingencia reales.

Sin embargo, el incremento técnico no está terminado hasta que se consolida documentalmente en la memoria global y se defiende con profesionalidad ante el cliente. Igual que hicimos al cierre del Sprint 1 y del Sprint 2, no podemos dejar documentos técnicos fragmentados y aislados.

*Hoy unificaremos el trabajo de las semanas 7 y 8 dentro de nuestro documento maestro `memoria.md` incorporando el **Capítulo 3**, cerraremos al 100 % el **Sprint Backlog 3**, estructuraremos el guion de la demostración oral de 3 minutos y marcaremos en nuestro repositorio la versión formal mediante la etiqueta `v0.3` utilizando la interfaz visual de IntelliJ IDEA con su correspondiente sincronización de tags».*

---

2. Fundamentos de consolidación y comunicación técnica



A. INTEGRACIÓN DOCUMENTAL ACUMULATIVA EN LA MEMORIA TÉCNICA

La memoria técnica del proyecto es un documento vivo que crece con cada sprint de forma incremental. Al finalizar el Sprint 3, se incorpora el **Capítulo 3 (Planificación temporal, gestión de riesgos y control de incidencias)** a continuación de los capítulos ya consolidados:

- **Capítulo 1 (Sprint 1).** Análisis de necesidades, problemática operativa del cliente, vinculación con ODS y viabilidad técnica inicial.
- **Capítulo 2 (Sprint 2).** Catálogo formal de requisitos funcionales y no funcionales, modelado visual de arquitectura y previsión de recursos.
- **Capítulo 3 (Sprint 3).** Secuenciación de fases (EDT/WBS), cronograma y dependencias (Gantt), matriz de evaluación de riesgos con planes de contingencia y procedimiento formal de registro y resolución de incidencias.

Integrar estos contenidos en `memoria.md` garantiza que cualquier revisor, cliente o tribunal disponga de un documento técnico único, coherente y trazable.

B. CRITERIO DE HECHO (*DEFINITION OF DONE* - DOD) Y CIERRE DE SPRINT

Antes de dar por concluido el sprint, cada tarea debe cumplir unos requisitos innegociables: * Código Java compilable sin advertencias ni dependencias rotas. * Algoritmos de control ejecutables con excepciones capturadas de forma preventiva. * Memoria técnica consolidada en `memoria.md` sin enlaces rotos ni inconsistencias de formato. * Pruebas manuales superadas y registradas en el cuaderno de incidencias.

C. ESTRUCTURA DE LA DEMOSTRACIÓN TÉCNICA EN 3 MINUTOS (*PITCH*)

Una defensa técnica ante un cliente o un tribunal evaluador debe ser precisa, estructurada y basada en hechos verificables:

- **Minuto 1. Contexto, hito alcanzado y cumplimiento temporal.** Presentar el proyecto, indicar en qué punto del cronograma se encuentra el equipo y destacar el principal riesgo mitigado durante el sprint.
- **Minuto 2. Demostración técnica en vivo (en consola).** Ejecutar la aplicación, forzar intencionadamente una entrada errónea para evidenciar que el control de excepciones y los bucles mantienen el programa estable sin caídas de ejecución.
- **Minuto 3. Sinergia técnica, lecciones aprendidas y próximos pasos.** Explicar cómo el depurador de IntelliJ IDEA permitió resolver incidencias en memoria y anticipar la transición hacia el Sprint 4 (diseño de clases propias y orientación a objetos).

D. SOPORTE VISUAL PARA LA EXPOSICIÓN: CÓDIGO DECLARATIVO Y HERRAMIENTAS DE ESCRITORIO

Para proyectar el soporte visual de la defensa se cuenta con dos vías de trabajo:

1. **Presentaciones declarativas basadas en código (Marp en Markdown).** Permite generar diapositivas directamente a partir de archivos Markdown delimitados por separadores `---`. Esta opción permite que la presentación se mantenga dentro del propio repositorio de Git y se visualice en cualquier visor web o extensión del editor de código.
2. **Presentaciones mediante software visual de escritorio (LibreOffice Impress).** Aplicación de software libre para la creación de diapositivas con soporte multimedia y esquemas gráficos. En entornos GNU/Linux puede ejecutarse mediante su versión portable en formato AppImage descargada en la carpeta del usuario (`~/bin/` o `~/Descargas/`), otorgando permisos de ejecución y lanzándola directamente sin requerir privilegios de superusuario, exportando la presentación en formato PDF en la carpeta de documentación.

E. ETIQUETADO VISUAL DE VERSIONES EN INTELLIJ IDEA (*SEMANTIC TAGGING*)

Al finalizar cada sprint, el repositorio debe reflejar el estado exacto del código y la documentación mediante una etiqueta anotada de Git:

- La etiqueta formal `v0.3` no requiere comandos de terminal; puede generarse de forma gráfica directamente desde el entorno de desarrollo mediante los diálogos de control de versiones de IntelliJ IDEA.
- Al enviar los cambios al repositorio remoto, se activa la opción de sincronización de etiquetas (*Push Tags*) para que la versión quede publicada inmediatamente en GitHub.

Laboratorio. Consolidación de la memoria técnica, cierre de backlog y demo v0.3

Laia Claramunt marca los pasos para la sesión práctica:

*«Es el momento de cerrar el Sprint 3. Cada estudiante abrirá su archivo **`memoria.md`**, añadirá al final el **Capítulo 3** aunando lo trabajado en `cronograma.md` y `riesgos-incidencias.md`, verificará y marcará al 100 % las tareas de su **Sprint Backlog 3**, redactará el **guion de la defensa técnica** de 3 minutos y sellará visualmente su entrega con la etiqueta **`v0.3`** en IntelliJ IDEA».*

PASO 1. CONSOLIDACIÓN DE LA MEMORIA TÉCNICA (`pi/docs/memoria.md`)

1. Abre tu proyecto en **IntelliJ IDEA**.
2. Abre el archivo maestro `azahartech/nombre-equipo/apellidos-nombre/pi/docs/memoria.md`.
3. Al final del documento (tras el Capítulo 2), añade el **Capítulo 3**, integrando el contenido de `cronograma.md` y `riesgos-incidencias.md` adaptado a **tu proyecto de la bolsa de proyectos**:

```
---

# Capítulo 3: Planificación temporal, gestión de riesgos y control de incidencias

## 3.1. Estructura de Descomposición del Trabajo (EDT)
El desarrollo del proyecto se desglosa en cuatro fases orientadas a entregables tangibles e independientes:

* **Fase 1: Análisis y Viabilidad (Semanas 1 a 6 - Sprints 1 y 2).** Definición operativa del problema, catálogo de requisitos y modelado de arquitectura IPO.
* **Fase 2: Construcción del Núcleo y Control de Calidad (Semanas 7 a 12 - Sprints 3 y 4).** Implementación de la lógica algorítmica, validación de datos, control de excepciones y auditoría con depurador.
* **Fase 3: Almacenamiento, Concurrencia e Interfaz Gráfica (Semanas 13 a 24 - Sprints 5 a 8).** Colecciones dinámicas, serialización y desarrollo de interfaces con Swing.
* **Fase 4: Consolidación y Pase a Producción (Semanas 25 a 30).** Pruebas de integración, empaquetado Maven y defensa pública.

---

## 3.2. Cronograma de proyecto y camino crítico

### Matriz de tareas y dependencias
| ID Tarea | Descripción del Paquete de Trabajo | Módulo | Duración | Dependencia | Hito Asociado |
| :---: | :--- | :---: | :---: | :---: | :---: |
| **T-01** | Catálogo de requisitos y arquitectura IPO | PI | 6 h | Ninguna | H1: Base aprobada |
| **T-02** | Configuración de entorno Maven y Git | ED | 9 h | Ninguna | H1: Base aprobada |
| **T-03** | Lógica con clases estándar Java | PR | 48 h | Ninguna | Incremento v0.2 |
| **T-04** | Cronograma de fases y matriz de riesgos | PI | 3 h | T-01 | H2: Planificación |
| **T-05** | Implementación de bucles y control condicional | PR | 24 h | T-03 | Incremento v0.3 |
| **T-06** | Pruebas de código y depuración con breakpoints | ED | 9 h | T-05 | Incremento v0.3 |
| **T-07** | POO modular y encapsulamiento | PR | 24 h | T-05 | Prototipo v0.4 |
| **T-08** | Versionado de ramas y empaquetado | ED | 9 h | T-06 | Prototipo v0.4 |
| **T-09** | Dossier técnico y defensa final Sprint 4 | PI | 3 h | T-04, T-07 | H3: Fin Trimestre 1 |

### Diagrama de Gantt
*(Incluye aquí el bloque de código Mermaid con el cronograma o la referencia a la imagen `img/cronograma.png` generada con GanttProject).*

---

## 3.3. Evaluación de riesgos y planes de contingencia

### Matriz de riesgos cuantificada
| ID | Categoría | Descripción del Riesgo | Prob. (1-5) | Imp. (1-5) | Severidad | Nivel |
| :---: | :---: | :--- | :---: | :---: | :---: | :---: |
| **RSK-01** | Técnico | Bloqueo o finalización abrupta por entrada no numérica en consola | 4 | 4 | **16** | Crítico |
| **RSK-02** | Técnico | Bucle infinito en la lógica de validación o espera | 3 | 4 | **12** | Medio |
| **RSK-03** | Organizativo | Desviación temporal en la entrega del incremento v0.3 | 3 | 3 | **9** | Medio |
| **RSK-04** | Alcance | Modificaciones de requisitos sin control previo de cambios | 2 | 4 | **8** | Medio |
| **RSK-05** | Técnico | Corrupción de datos por valores fuera de rango admisible | 3 | 5 | **15** | Crítico |

*(Incluye aquí el cuadrante declarativo en Mermaid o la referencia a la imagen `img/matriz-riesgos.png` exportada desde Draw.io Desktop).*

### Medidas de prevención y planes de contingencia operativos
* **Plan para RSK-01 (Entradas anómalas):** Captura preventiva mediante bloques `try-catch` y reintento guiado dentro de un bucle `while`. Si la entrada es inválida, se muestra aviso pedagógico y se restaura el flujo sin caída de la JVM.
* **Plan para RSK-02 (Bucles infinitos):** Incorporación de contadores de seguridad (*guard clauses*) con límite de intentos y depuración paso a paso en IntelliJ IDEA inspeccionando variables centinela.
* **Plan para RSK-03 (Desviación en cronograma):** Priorización estricta de requisitos esenciales frente a mejoras cosméticas para salvaguardar la fecha comprometida.

---

## 3.4. Control de calidad y protocolo de incidencias técnicas

### Procedimiento formal de seguimiento
Toda anomalía detectada en fase de pruebas o depuración sigue el ciclo: *Identificada* $\rightarrow$ *Registrada* $\rightarrow$ *En depuración* $\rightarrow$ *Resuelta* $\rightarrow$ *Cerrada*.

### Registro histórico de incidencias del sprint
* **Incidencia INC-01:** Entrada vacía o salto de línea prematuro en la lectura por consola provoca `NoSuchElementException`.
  * **Causa técnica:** Llamada directa a lectura sin comprobación de fin de línea.
  * **Solución aplicada:** Validación de búfer mediante `hasNextLine()` y verificación de cadena no vacía antes de procesar el dato.
  * **Verificación:** Batería de 10 pruebas sucesivas superadas con entradas límite y caracteres no imprimibles.

---

## 3.5. Registro acumulativo de Inteligencia Artificial (Prompt Log)

| Fecha | Herramienta | Objetivo | Prompt resumido | Ajuste crítico aplicado |
| :---: | :---: | :--- | :--- | :--- |
| 13/11/2026 | ChatGPT / Claude | Contrastar estructura de mitigación de riesgos | *"Revisa este plan de contingencia para excepciones en consola Java..."* | Se adaptó la respuesta a las clases estándar disponibles en Java SE 21 sin recurrir a dependencias externas. |
```

PASO 2. REDACCIÓN DEL GUION DE LA DEMOSTRACIÓN ORAL (`pi/docs/guion-demo-v03.md`)

1. En la carpeta `azahartech/nombre-equipo/apellidos-nombre/pi/docs/` , crea el archivo `guion-demo-v03.md` .
2. Completa la estructura de exposición cronometrada en **3 minutos**:

```
# Guion de la Sprint Review — Incremento v0.3
**Proyecto:** [nombre de tu proyecto]
**Ponente:** [tu nombre]
```

```
**Duración límite:** 180 segundos (3 minutos)
**Fecha de defensa:** 13 de noviembre de 2026

---

## Minuto 1: Contexto, cronograma y riesgos (00:00 - 01:00)
* **Apertura:** Saludo profesional y presentación del proyecto en el marco de AzaharTech.
* **Hito de planificación:** Mostrar brevemente el cronograma y confirmar que el proyecto cumple con las fechas estimadas para el Sprint 3.
* **Gestión de riesgos:** Explicar cómo se detectó el riesgo de caídas por datos erróneos y la estrategia de mitigación adoptada.

## Minuto 2: Demostración técnica en vivo (01:00 - 02:00)
* **Ejecución en consola:** Lanzar la aplicación desde IntelliJ IDEA.
* **Caso de uso nominal:** Introducir datos válidos y mostrar la respuesta correcta del algoritmo.
* **Caso de prueba límite:** Introducir texto aleatorio o pulsar *Enter* repetidamente.
* **Evidencia de robustez:** Demostrar que el programa no cae, captura la excepción, avisa al usuario y permite reintentar la operación limpiamente.

## Minuto 3: Sinergia técnica y próximos pasos (02:00 - 03:00)
* **Depuración e incidencias:** Explicar cómo el uso de puntos de ruptura en Entornos de Desarrollo permitió resolver la incidencia `INC-01`.
* **Próximo incremento (Sprint 4):** Avanzar el salto hacia la orientación a objetos propia (creación de clases, atributos privados y constructores).
* **Cierre:** Agradecer la atención y quedar a disposición para responder preguntas técnicas.
```

PASO 3. CIERRE AL 100 % DEL SPRINT BACKLOG 3 (`pi/backlog/sprint3-backlog.md`)

1. Abre el archivo `pi/backlog/sprint3-backlog.md` .
2. Verifica que todas las tareas están finalizadas y marca la última tarea de consolidación con `[x]` :

```
# Sprint Backlog 3 — [nombre de tu proyecto]
**Periodo:** 26 de octubre – 13 de noviembre de 2026
**Responsable:** [tu nombre]
**Estado general:** Completado al 100 %

## Objetivo general del Sprint 3
Construir el incremento v0.3: definir la secuenciación temporal y matriz de riesgos en Proyecto Intermodular, dominar el uso de puntos de ruptura y
seguimiento de variables en Entornos de Desarrollo, e implementar estructuras de control condicionales y repetitivas robustas en Programación.

---

## Tareas

### Módulo: Proyecto Intermodular (PI)
- [x] T-PI-10: Elaborar la EDT, matriz de dependencias y Diagrama de Gantt (`pi/docs/cronograma.md` o `pi/docs/img/cronograma.png`)
- [x] T-PI-11: Registrar el uso ético de IA (*Prompt Log*) en la planificación temporal
- [x] T-PI-12: Redactar la matriz de riesgos técnicos y planes de contingencia (`pi/docs/riesgos-incidencias.md` o `pi/docs/img/matriz-riesgos.png`)
- [x] T-PI-13: Consolidar el Capítulo 3 de la memoria y ensayar la demo técnica v0.3 (`pi/docs/memoria.md` y `pi/docs/guion-demo-v03.md`)

### Módulo: Entornos de Desarrollo (ED)
- [x] T-ED-08: Diseñar casos de prueba de caja negra aplicando particiones de equivalencia y valores límite
- [x] T-ED-09: Configurar puntos de ruptura (*breakpoints*) condicionales en IntelliJ IDEA para inspección de memoria
- [x] T-ED-10: Documentar el procedimiento formal de recogida y registro de incidencias técnicas en coordinación con PI

### Módulo: Programación (PR)
- [x] T-PR-07: Implementar lógica condicional mediante bloques `if / else if / else` y selector `switch`
- [x] T-PR-08: Construir bucles iterativos controlados (`while`, `for`) para la captura repetitiva de datos
- [x] T-PR-09: Incorporar gestión de excepciones básica con bloques `try-catch` frente a entradas erróneas del usuario
```

PASO 4. CONFIRMACIÓN, ETIQUETADO VISUAL Y SINCRONIZACIÓN EN GITHUB

1. En IntelliJ IDEA, abre la ventana de confirmación (`Ctrl + K`).
2. Asegúrate de incluir los archivos modificados y creados:

- `pi/docs/memoria.md`
- `pi/docs/guion-demo-v03.md`
- `pi/backlog/sprint3-backlog.md`

3. Escribe el mensaje de confirmación estandarizado:

```
docs(pi): consolidar capitulo 3 en memoria, guion de demo oral y cerrar sprint backlog 3 al 100%
```

1. Pulsa en **Commit** (o en la flecha contigua seleccionando únicamente *Commit* para registrar el estado local antes de crear la etiqueta).
2. Crea la etiqueta visualmente desde IntelliJ IDEA:

- En el menú superior, accede a **Git** $\rightarrow$ **New Tag...** (o abre la pestaña inferior **Git / Log**, haz clic derecho sobre el commit recién creado y selecciona **New Tag...**).
- En el campo **Tag name**, escribe exactamente: `v0.3`.
- En el campo **Message**, introduce la descripción del incremento:
  `Incremento v0.3: Planificacion temporal, matriz de riesgos, estructuras de control y depuracion formal`
- Haz clic sobre el botón **Create Tag**.

3. Sincroniza el commit y la etiqueta con el repositorio remoto:

- Abre la ventana de envío de cambios mediante el atajo `Ctrl + Shift + K` (o desde el menú **Git** $\rightarrow$ **Push...**).
- En la esquina inferior izquierda del diálogo de confirmación, marca la casilla de verificación **Push Tags** (asegurándote de que indica *All* o la etiqueta `v0.3`).
- Haz clic en el botón **Push**.

4. Accede a tu repositorio en GitHub desde el navegador, entra en la sección **Tags / Releases** para comprobar que la versión `v0.3` aparece publicada y verifica que `memoria.md` muestra los Capítulos 1, 2 y 3 consolidados con todas sus tablas y diagramas visibles.

3. Sprint 2. Especificación de requisitos del sistema

3.1 Semana 4. Catálogo de requisitos y Sprint Backlog 2

3.1.1 Día 4 - 1 sesión

Teoría. Requisitos funcionales y no funcionales

1. Caso guía en AzaharTech

Es viernes por la tarde en la sede de **AzaharTech** en Castellón de la Plana. Tras haber cerrado con éxito el Sprint 1 y haber congelado la primera entrega en GitHub, el equipo arranca el **Sprint 2**.

En la sala de reuniones, **Laia Claramunt** proyecta un nuevo documento en la pantalla. A su lado, **Alba Torres** revisa las especificaciones del entorno Maven y **Pau Ferrer** tiene abierto el esquema del vestíbulo del **IES El Caminàs**.

Laia toma la palabra dirigiéndose al equipo de trabajo:

*«En el Sprint 1 definimos el problema global, identificamos a los actores y demostramos que la lógica base era viable. Pero un cliente no puede firmar un contrato sobre intenciones generales; necesita saber con exactitud matemática **qué hará y qué no hará el software**.*

Si el equipo directivo del IES El Caminàs nos dice 'queremos que el sistema sea rápido y seguro', esa frase no le sirve a un programador. ¿Qué significa 'rápido'? ¿Menos de un segundo? ¿Menos de diez? ¿Qué significa 'seguro'? ¿Qué datos valida?

*Hoy aprenderemos a transformar las necesidades del cliente en un **Catálogo Formal de Requisitos del Sistema**. Aprenderemos a distinguir los **Requisitos Funcionales (RF)** de los **Requisitos No Funcionales (RNF)**, contrastaremos la documentación técnica oficial para no basarnos en suposiciones y abriremos el **Sprint Backlog 2** que guiará nuestro trabajo durante las próximas tres semanas».*

2. Fundamentos de ingeniería de requisitos

EL PUENTE ENTRE EL CLIENTE Y EL CÓDIGO

1. Necesidad del Cliente (Lenguaje natural)	2. Requisito Funcional (RF) (¿Qué hace el sistema?)	3. Requisito No Funcional (¿Bajo qué restricciones?)
"Queremos que los chicos fichen rápido al entrar"	El sistema generará un código de acceso único con el DNI y la marca de tiempo.	La respuesta de validación en pantalla debe realizarse en un tiempo inferior a 1,5 s.

A. REQUISITOS FUNCIONALES FRENTE A REQUISITOS NO FUNCIONALES

Para que el equipo de desarrollo pueda construir el software sin ambigüedades, los requisitos se dividen en dos categorías complementarias:

- **Requisitos Funcionales (RF).** Describen los servicios, comportamientos y funciones operativas que debe ejecutar el software ante determinadas entradas. Responden a la pregunta: *¿Qué hace el sistema?*
- *En el caso guía (IES El Caminàs):*
- **RF-01.** El sistema debe capturar el identificador del alumno y los minutos lectivos programados.
- **RF-02.** El sistema debe componer un código de verificación que una el prefijo del centro, el DNI y el número de terminal.
- **Requisitos No Funcionales (RNF).** Definen las restricciones técnicas, propiedades de calidad, rendimiento o normativas que limitan la solución. Responden a la pregunta: *¿Bajo qué condiciones o restricciones debe operar?*
- *En el caso guía (IES El Caminàs):*
- **RNF-01 (Tecnología).** La aplicación debe ejecutarse sobre la Máquina Virtual de Java utilizando OpenJDK 21 LTS.
- **RNF-02 (Usabilidad/Presentación).** La salida de datos por consola debe estar estructurada de forma tabular con decimales acotados.
- **RNF-03 (Mantenibilidad).** El código debe estructurarse mediante proyectos gestionados con Apache Maven y control de dependencias.

B. REGLAS DE REDACCIÓN DE REQUISITOS PROFESIONALES

Un requisito técnico mal redactado propaga errores en cascada hacia las fases de diseño y pruebas. En AzaharTech aplicamos la regla **SMART**:

1. **Identificador unívoco.** Todo requisito comienza con un código (`RF-01`, `RF-02`, `RNF-01`).
2. **Sin ambigüedades.** Se evitan términos subjetivos como *«fácil»*, *«rápido»*, *«eficiente»* o *«moderno»*. Se utilizan verbos operativos (*«El sistema calculará...»*, *«El sistema almacenará...»*).
3. **Verificable.** Debe ser posible diseñar una prueba técnica objetiva que demuestre si el requisito se cumple o no.

C. BÚSQUEDA Y CONTRASTE DE FUENTES TÉCNICAS OFICIALES

Para especificar los requisitos de un proyecto no podemos recurrir a foros informales, blogs desactualizados o código no mantenido:

- **Fuentes primarias y oficiales.** Se consultan las especificaciones oficiales de Oracle para Java SE 21, la documentación de la Apache Software Foundation para Maven y las guías de estándares de la industria (IEEE 830 / ISO 25010).
- **Detección de información no contrastada.** Antes de dar por válido un requisito o librería externa, se verifica su licencia, su mantenimiento activo y su compatibilidad con la JVM.

D. EL USO ÉTICO DE LA INTELIGENCIA ARTIFICIAL COMO ASISTENTE PARA DOCUMENTAR REQUISITOS

La IA generativa se utiliza como asistente para desglosar requisitos y anticipar casos límite que el analista haya podido pasar por alto:

- **Uso correcto.** Pedir a la IA que revise una lista inicial de requisitos en busca de ambigüedades o incoherencias.
- **El Prompt Log del Sprint 2.** Cada consulta debe quedar documentada en la memoria, detallando qué aportación de la IA se ha aceptado y qué partes se han corregido para adaptarse a la realidad técnica del proyecto.

Laboratorio. Catálogo de requisitos y Sprint Backlog 2

Laia Claramunt asigna las tareas de la sesión:

*«Durante los próximos minutos, cada uno de vosotros abrirá el archivo de requisitos de su proyecto propio de la bolsa de proyectos. Definiréis al menos **cuatro Requisitos Funcionales** y **dos Requisitos No Funcionales**, contrastaréis las fuentes oficiales y crearéis el archivo `sprint2-backlog.md` que coordinará las tres semanas que tenemos por delante».*

PASO 1. CREACIÓN DEL ARCHIVO DE CATÁLOGO DE REQUISITOS (`pi/docs/requisitos-sistema.md`)

1. Abre tu proyecto en **IntelliJ IDEA**.
2. Dentro de tu carpeta `azahartech/nombre-equipo/apellidos-nombre/pi/docs/`, crea el archivo `requisitos-sistema.md`.
3. Redacta el catálogo aplicando la siguiente plantilla formal adaptada a **tu proyecto elegido de la bolsa de proyectos**:

```
# Catálogo de requisitos del sistema y fuentes técnicas
**Consultora:** AzaharTech Software Consulting
**Proyecto Seleccionado:** [nombre de tu proyecto de la bolsa de proyectos]
**Cliente / Sector:** [nombre de la empresa o sector profesional]
**Desarrollador/a:** [tus apellidos, tu nombre]
**Fecha:** 9 de octubre de 2026
**Versión:** 1.0 (Sprint 2)

---

## 1. Alcance y objetivos técnicos del Sprint 2
El objetivo de esta fase es estructurar la lógica del software mediante el uso de objetos estándar y clases predefinidas de Java, gestionando la construcción mediante Apache Maven y definiendo con exactitud el comportamiento esperado del sistema.

---

## 2. Catálogo de Requisitos Funcionales (RF)
| Código | Nombre del Requisito | Descripción técnica obligatoria | Prioridad |
| :---: | :--- | :--- | :---: |
| **RF-01** | Captura de parámetros base | El sistema solicitará interactivamente los identificadores y valores iniciales de la operación del cliente. | Alta |
|
**RF-02** | Procesamiento de objetos | El sistema empleará clases predefinidas del lenguaje (Scanner, String, Math) para transformar los datos de entrada. | Alta |
| **RF-03** | Generación de identificador | El sistema compondrá un identificador o código alfanumérico combinando prefijos fijos y marcas de transacción. | Media |
| **RF-04** | Resumen estructurado | El sistema emitirá un informe de resultados formateado sin pérdida de precisión numérica. | Media |

---

## 3. Catálogo de Requisitos No Funcionales (RNF)
| Código | Categoría | Restricción técnica o estándar | Verificación |
| :---: | :--- | :--- | :--- |
| **RNF-01** | Entorno y plataforma | El software debe compilar y ejecutarse bajo OpenJDK 21 LTS sin librerías externas no estándar. | Inspección de compilación |
| **RNF-02** | Gestión de proyecto | La estructura de carpetas y dependencias debe regirse bajo el estándar de Apache Maven con su archivo `pom.xml`. | Archivo de configuración |
| **RNF-03** | Calidad de código | Nombres de variables en camelCase, constantes en UPPER_SNAKE_CASE y código autoformateado. | Revisión de estilo |

---

## 4. Fuentes técnicas oficiales contrastadas
1. **Oracle Java SE 21 Specification:** Documentación de la API estándar para clases del paquete `java.lang` y `java.util`. URL: `https://docs.oracle.com/en/java/javase/21/`
2. **Apache Maven Project Standard Directory Layout:** Especificación oficial de la jerarquía de carpetas en proyectos de software. URL: `https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html`

---

## 5. Registro ético de uso de Inteligencia Artificial (Prompt Log)
| Fecha | Herramienta | Objetivo de la consulta | Prompt introducido | Revisión crítica y ajuste aplicado |
| :---: | :---: | :--- | :--- | :--- |
| 09/10/2026 | ChatGPT / Claude | Redactar requisitos no funcionales | *"Actúa como analista y redacta 2 requisitos no funcionales SMART para..."* | Se adaptaron las métricas de rendimiento y se eliminaron peticiones de bases de datos externas para respetar el Sprint 2. |
```

PASO 2. CREACIÓN DEL SPRINT BACKLOG 2 (`pi/backlog/sprint2-backlog.md`)

1. En la carpeta `pi/backlog/`, crea el archivo `sprint2-backlog.md`.
2. Define la lista de tareas del **Sprint 2 (5 oct – 23 oct)** desglosada para los tres módulos:

```
# Sprint Backlog 2 — [nombre de tu proyecto]
**Periodo:** 5 de octubre – 23 de octubre de 2026
**Responsable:** [tu nombre]

## Objetivo general del Sprint 2
Construir el incremento v0.2: formalizar el catálogo de requisitos del sistema, estructurar el proyecto con Apache Maven en Entornos de Desarrollo y aplicar clases y objetos predefinidos en Programación.

---

## Tareas

### Módulo: Proyecto Intermodular (PI)
- [ ] T-PI-06: Redactar el catálogo de requisitos funcionales y no funcionales (`pi/docs/requisitos-sistema.md`)
- [ ] T-PI-07: Contrastar fuentes técnicas oficiales y registrar el *Prompt Log*
- [ ] T-PI-08: Modelar el diagrama de bloques funcional en Draw.io (Semana 5)
- [ ] T-PI-09: Actualizar el avance y planificar el seguimiento del sprint (Semana 6)
```

```
### Módulo: Entornos de Desarrollo (ED)
- [ ] T-ED-05: Comprender el ciclo de compilación a Bytecode y la virtualización en la JVM
- [ ] T-ED-06: Crear y configurar la estructura de proyecto con Apache Maven (`pom.xml`)
- [ ] T-ED-07: Configurar y verificar el archivo `.gitignore` para exclusiones de compilación de Maven (`target/`)

### Módulo: Programación (PR)
- [ ] T-PR-04: Instanciar y utilizar objetos de clases predefinidas (`Scanner`, `String`, `Math`, `Random`)
- [ ] T-PR-05: Programar llamadas a métodos estáticos y métodos de instancia pasando parámetros
- [ ] T-PR-06: Integrar los cálculos y generación de cadenas en el código guía del proyecto propio
```

PASO 3. CONFIRMACIÓN Y SINCRONIZACIÓN EN GITHUB

1. Abre el panel **Commit** en IntelliJ IDEA (`Ctrl + K`).
2. Comprueba que aparecen seleccionados `pi/docs/requisitos-sistema.md` y `pi/backlog/sprint2-backlog.md`.
3. Escribe el mensaje siguiendo el estándar de **Conventional Commits**:

```
docs(pi): definir catalogo de requisitos del sistema y abrir sprint backlog 2
```

4. Haz clic en **Commit and Push** y verifica en GitHub que ambos archivos quedan registrados.

3.2 Semana 5. Modelado visual y previsión de recursos

3.2.1 Día 5 - 1 sesión

Teoría. Modelado visual y tratamiento de imágenes

1. Caso guía en AzaharTech

Es viernes 16 de octubre por la tarde en la sede de **AzaharTech** en Castellón de la Plana. El equipo de trabajo se reúne para la segunda sesión del Sprint 2.

En la pantalla digital de la sala, **Laia Claramunt** proyecta el catálogo de requisitos que el equipo aprobó la semana anterior. A su lado, **Alba Torres** tiene abierta en su portátil la herramienta **Draw.io**, donde ha transformado los requisitos del **IES El Caminàs** en un esquema visual nítido y estructurado:

```
+----------------------+      +--------------------------+      +----------------------+
|  CAPTURA Y TERMINAL   | ===> |   LÓGICA CENTRAL JAVA    | ===> |  VISUALIZACIÓN Y LOG  |
|  - Lector móvil       |      |   - Paquete java.lang    |      |  - Pantalla vestíbulo |
|  - Escáner de teclado |      |   - Clases Math y String |      |  - Consola formateada |
+----------------------+      +--------------------------+      +----------------------+
```

Laia toma la palabra dirigiéndose a **Pau Ferrer** y al estudiante:

«En la semana anterior redactamos los Requisitos Funcionales y No Funcionales. Pero cuando un cliente o un comité técnico revisa una memoria, enfrentarse a páginas de texto denso sin apoyo gráfico dificulta entender cómo encajan las piezas del sistema.

*Un ingeniero de software debe saber traducir las especificaciones escritas a **modelos visuales de arquitectura**. Para el IES El Caminàs, este diagrama de bloques permite que el jefe de estudios entienda exactamente por dónde viajan los datos y qué módulos intervienen.*

*Hoy aprenderemos a utilizar una herramienta estándar de diagramación técnica (**Draw.io / diagrams.net**), aplicaremos el **tratamiento e inserción formal de imágenes técnicas** en nuestra documentación Markdown y cuantificaremos la **previsión de recursos materiales y humanos** que exige nuestro proyecto».*

---

2. Diagramas de arquitectura y estimacion técnica

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      DEL REQUISITO AL MODELO VISUAL (IPO)                │
├──────────────────────────┬──────────────────────────┬────────────────────┤
│ 1. Subsistema de Entrada │ 2. Subsistema de Proceso │ 3. Subsistema de Salida │
├──────────────────────────┼──────────────────────────┼────────────────────┤
│ Qué componentes reciben  │ Qué clases, métodos y    │ Qué dispositivos o archivos │
│ los datos del exterior   │ algoritmos transforman   │ reciben la información │
│ (dispositivos, teclado). │ los datos en memoria.    │ procesada final.   │
└──────────────────────────┴──────────────────────────┴────────────────────┘
```

A. EL DIAGRAMA DE BLOQUES FUNCIONAL EN LA INGENIERÍA DEL SOFTWARE

El Diagrama de Bloques Funcional es una técnica de modelado de alto nivel que representa la arquitectura de un sistema dividiéndolo en bloques funcionales interconectados:

- **Cajas negras funcionales.** Cada bloque representa un subsistema con una responsabilidad única y bien delimitada (captura, procesamiento matemático, formateo).
- **Flechas de flujo.** Indican la dirección en la que viajan los datos y qué tipo de información se transmite entre subsistemas.
- **Independencia de implementación.** El diagrama muestra *cómo colaboran las partes*, permitiendo que cualquier desarrollador entienda el flujo antes de inspeccionar el código Java o la estructura de Maven.

B. HERRAMIENTAS TIC PARA DIAGRAMACIÓN TÉCNICA: DRAW.IO (DIAGRAMS.NET)

Ya no dibujaremos esquemas de ingeniería a mano alzada ni con herramientas de retoque fotográfico genéricas:

- **Draw.io (diagrams.net).** Es una herramienta estándar, libre y multiplataforma especializada en diagramación técnica.
- Permite alinear elementos con rejilla milimétrica, utilizar conectores ortogonales inteligentes y exportar esquemas en formatos vectoriales o de alta resolución listos para documentación técnica.

C. TRATAMIENTO FORMAL DE IMÁGENES TÉCNICAS EN MARKDOWN

Una memoria técnica profesional no puede incrustar capturas borrosas o con fondos descuadrados. Al incorporar diagramas a la documentación se deben respetar tres normas de calidad:

1. **Exportación optimizada.** Las imágenes deben exportarse en formato **PNG** (con fondo transparente o blanco nítido) o **SVG** (vectorial escalable), recortando los márgenes sobrantes (*crop*).
2. **Rutas relativas estrictas.** En Markdown, las imágenes deben guardarse en una subcarpeta dedicada (`pi/docs/img/`) y enlazarse mediante rutas relativas (`![Descripción](img/diagrama.png)`), garantizando que se visualicen correctamente tanto en local como en GitHub.
3. **Pie de figura numerado.** Toda imagen técnica debe ir acompañada de un pie explicativo numerado («*Figura 1. Diagrama de bloques funcional de la arquitectura del sistema*»).

D. PREVISIÓN Y JUSTIFICACIÓN DE RECURSOS

Un proyecto de software viable exige cuantificar con rigor los recursos necesarios para su ejecución:

- **Recursos humanos.** Estimación de horas de trabajo por desarrollador y asignación de responsabilidades.
- **Recursos hardware.** Equipamiento físico requerido tanto para el desarrollo (puestos de trabajo con 8 GB de RAM) como para el despliegue en el cliente (pantallas, servidores, terminales de red).
- **Recursos software.** Licencias y herramientas necesarias (OpenJDK 21 LTS, IntelliJ IDEA Community, Git, GitHub y librerías del ecosistema Maven).

Laboratorio. Modelado visual y prevision de recursos

Laia Claramunt asigna las tareas de la sesión:

*«Durante los próximos minutos, cada uno de vosotros abrirá Draw.io en su navegador y diseñará el **Diagrama de Bloques Funcional de su proyecto de la bolsa de proyectos**. Exportaréis la imagen a vuestra carpeta de documentación, redactaréis la memoria técnica de arquitectura y recursos (**`arquitectura-recursos.md`**) y actualizaréis el avance semanal en vuestro **Sprint Backlog 2**».*

PASO 1. DISEÑO Y EXPORTACIÓN DEL DIAGRAMA DE BLOQUES EN DRAW.IO

1. Abre tu navegador web y accede a app.diagrams.net.
2. Selecciona **Crear nuevo diagrama** (Diagrama en blanco).
3. Modela la arquitectura funcional de tu proyecto propio siguiendo el patrón de tres subsistemas:

- **Bloque 1 (Entrada de datos).** Dibuja los componentes que capturan la información de tu cliente (por ejemplo, teclado de terminal, escáner, sensores o parámetros).
- **Bloque 2 (Procesamiento lógico Java).** Dibuja el núcleo de la aplicación indicando las clases estándar que intervienen (`Scanner` , `String` , `Math`).
- **Bloque 3 (Salida de resultados).** Dibuja los soportes donde se muestra o registra la información procesada (consola formateada, pantalla del usuario, comprobante de operación).

4. Conecta los bloques mediante flechas de flujo señalando qué dato viaja entre ellos.
5. Ajusta el diseño: aplica colores sobrios corporativos, textos nítidos y comprueba que la ortografía sea correcta.
6. Exporta el diagrama:

- Menú superior: **Archivo -> Exportar como -> PNG...**
- En las opciones, marca la casilla **Recortar** (*Crop*) para eliminar espacios en blanco sobrantes alrededor del diagrama.
- Guarda el archivo con el nombre: `diagrama-bloques.png` .

PASO 2. INTEGRACIÓN DE LA IMAGEN Y REDACCIÓN TÉCNICA (`pi/docs/arquitectura-recursos.md`)

1. En tu proyecto de IntelliJ, crea la carpeta `img/` dentro de `pi/docs/` si no existe: `azahartech/nombre-equipo/apellidos-nombre/pi/docs/img/` .
2. Mueve el archivo `diagrama-bloques.png` dentro de `pi/docs/img/` .
3. Dentro de `pi/docs/` , crea el archivo `arquitectura-recursos.md` .
4. Redacta el documento aplicando la siguiente plantilla formal adaptada a **tu proyecto propio**:

```
# Arquitectura funcional del sistema y previsión de recursos
**Consultora:** AzaharTech Software Consulting
**Proyecto Seleccionado:** [nombre de tu proyecto de la bolsa de proyectos]
**Desarrollador/a:** [tus apellidos, tu nombre]
**Fecha:** 16 de octubre de 2026
**Versión:** 1.0 (Sprint 2)

---

## 1. Diagrama de Bloques Funcional de la Arquitectura
El siguiente esquema técnico modela visualmente el flujo de información y la descomposición en subsistemas de la solución:

![Diagrama de Bloques Funcional](img/diagrama-bloques.png)
*Figura 1. Modelo de arquitectura funcional Entrada-Procesamiento-Salida (IPO).*

---

## 2. Descripción de los Subsistemas
* **Subsistema de Entrada (Captura).** [Describe qué componentes capturan los datos de tu cliente y mediante qué mecanismos].
* **Subsistema de Procesamiento (Lógica Java).** [Explica cómo las clases Java y las operaciones matemáticas transforman los datos en memoria].
* **Subsistema de Salida (Presentación).** [Describe el formato y soporte final donde se presentan los resultados al usuario].

---

## 3. Previsión de Recursos Materiales y Humanos
| Categoría de Recurso | Descripción del Elemento | Justificación Técnica en el Proyecto |
| :--- | :--- | :--- |
| **Recursos Humanos** | Desarrollador/a junior (34 h lectivas) | Análisis, documentación en PI, codificación en PR y control en ED. |
| **Hardware de Desarrollo** | PC con procesador x86_64 y 8 GB RAM | Entorno de compilación local y ejecución fluida de IntelliJ IDEA. |
| **Hardware de Despliegue** | Terminal de cliente / Pantalla operativa | Soporte físico para la visualización del software en producción. |
| **Software de Base** | OpenJDK 21 LTS | Entorno de ejecución estándar multiplataforma mediante la JVM. |
| **Herramientas de Gestión** | Apache Maven 3.9+ / Git & GitHub | Automatización de construcción y control de versiones distribuido. |
```

PASO 3. ACTUALIZACIÓN DEL ESTADO DEL SPRINT BACKLOG 2 (`pi/backlog/sprint2-backlog.md`)

Abre tu archivo `sprint2-backlog.md` y actualiza el estado de las tareas de los tres módulos al cierre de la segunda semana del sprint:

```
# Sprint Backlog 2 — [nombre de tu proyecto]
**Estado al cierre de la Semana 5 (16 de octubre de 2026)**

## Tareas

### Módulo: Proyecto Intermodular (PI)
- [x] T-PI-06: Redactar el catálogo de requisitos funcionales y no funcionales (`pi/docs/requisitos-sistema.md`)
- [x] T-PI-07: Contrastar fuentes técnicas oficiales y registrar el *Prompt Log*
- [x] T-PI-08: Modelar el diagrama de bloques funcional en Draw.io (`pi/docs/arquitectura-recursos.md`)
- [ ] T-PI-09: Consolidar el Capítulo 2 de la memoria y preparar la demo del sprint (Semana 6)

### Módulo: Entornos de Desarrollo (ED)
- [x] T-ED-05: Comprender el ciclo de compilación a Bytecode y la virtualización en la JVM
- [x] T-ED-06: Crear y configurar la estructura de proyecto con Apache Maven (`pom.xml`)
- [ ] T-ED-07: Configurar y verificar el archivo `.gitignore` para exclusiones de compilación de Maven (`target/`)

### Módulo: Programación (PR)
- [x] T-PR-04: Instanciar y utilizar objetos de clases predefinidas (`Scanner`, `String`, `Math`, `Random`)
- [x] T-PR-05: Programar llamadas a métodos estáticos y métodos de instancia pasando parámetros
- [ ] T-PR-06: Integrar los cálculos y generación de cadenas en el código guía del proyecto propio
```

PASO 4. CONFIRMACIÓN Y SINCRONIZACIÓN EN GITHUB

1. Abre el panel **Commit** en IntelliJ IDEA (`Ctrl + K`).
2. Comprueba que aparecen seleccionados `pi/docs/arquitectura-recursos.md` y `pi/docs/img/diagrama-bloques.png` .
3. Escribe el mensaje siguiendo el estándar de **Conventional Commits**:

```
docs(pi): modelar diagrama de bloques funcional en drawio y estimar recursos
```

4. Haz clic en **Commit and Push** y verifica en GitHub que ambos archivos quedan registrados.

3.3 Semana 6. Consolidación de la memoria y demo técnica v0.2

3.3.1 Día 6 - 1 sesión

Teoría. Verificación formal y defensa del incremento

1. Caso guía en AzaharTech

Es viernes 23 de octubre por la tarde en la sede de **AzaharTech** en Castellón de la Plana. Las tres semanas del segundo ciclo de desarrollo concluyen hoy.

En la sala de reuniones principal, **Laia Claramunt** proyecta el documento técnico consolidado del caso guía del **IES El Caminàs**. En pantalla se observa cómo el catálogo de requisitos redactado en la primera semana y el diagrama de bloques diseñado en Draw.io en la segunda semana se han ensamblado de forma armoniosa dentro de una memoria técnica unificada con numeración jerárquica y estilo corporativo impecable.

A su lado, **Alba Torres** tiene abierta la estructura de Apache Maven en IntelliJ y **Pau Ferrer** cronometra una presentación de prueba de tres minutos.

Laia toma la palabra dirigiéndose a los desarrolladores del equipo de trabajo:

«En las dos semanas anteriores definimos el catálogo de requisitos funcionales y modelamos la arquitectura funcional en Draw.io. En Programación habéis dominado el uso de objetos estándar como `Scanner`, `Math` y `String`, y en Entornos de Desarrollo habéis configurado la jerarquía estándar de Maven.

*Pero un proyecto de ingeniería no se entrega con notas sueltas dispersas por el disco duro. En AzaharTech, al finalizar cada ciclo de desarrollo, integramos todos los avances en un capítulo consolidado de la **Memoria del Proyecto**, auditamos que el **Sprint Backlog esté cerrado al 100 % de cumplimiento** y preparamos la **defensa oral de la solución técnica ante el cliente**.*

*Hoy cerraremos formalmente el **Capítulo 2 de la memoria técnica de vuestro proyecto de la bolsa de proyectos**, comprobaremos que todos los requisitos y recursos están justificados sin ambigüedades y ensayaremos el guion de comunicación técnica para la Sprint Review del incremento v0.2».*

---

2. Control de calidad y defensa oral

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        EL CIERRE DOCUMENTAL DEL SPRINT 2                 │
├──────────────────────────┬──────────────────────────┬────────────────────┤
│ 1. Consolidación         │ 2. Cierre del Backlog    │ 3. Defensa oral de requisitos │
│    (Capítulo 2 memoria)  │    (Definition of Done)  │    (The 3-Minute Review)      │
├──────────────────────────┼──────────────────────────┼────────────────────┤
│ Se ensamblan requisitos, │ Se audita que todas las  │ Guion estructurado: requisitos│
│ bloques Draw.io, recursos│ tareas de PR, ED y PI están │ clave -> Modelo visual IPO -> │
│ y fuentes en un documento│ verificadas y marcadas:  │ Viabilidad y tecnologías base.│
│ formal normalizado.      │ - [x] 100% completado    │                    │
└──────────────────────────┴──────────────────────────┴────────────────────┘
```

A. LA CONSOLIDACIÓN DOCUMENTAL COMO GARANTÍA DE INGENIERÍA

Tener requisitos en un archivo, diagramas en una carpeta de imágenes y notas de recursos en borradores genera dispersión y desconfianza en el cliente.

- **El principio del documento maestro.** Al cierre de cada sprint, todos los elementos técnicos se integran en un documento consolidado bajo normas de estilo estrictas:

a. Numeración jerárquica homogénea (1, 1.1, 1.2, 2, 2.1...).

b. Tablas estandarizadas con identificadores unívocos (`RF-01`, `RNF-01`).

c. Figuras técnicas incrustadas con pie numerado explicativo.

d. Referencias bibliográficas a fuentes técnicas oficiales (estilo IEEE/APA).

B. CRITERIOS DE VERIFICACIÓN Y CIERRE DEL SPRINT BACKLOG 2 (RA4.A)

Antes de dar por cerrado el sprint, el estudiante debe contrastar que los entregables cumplen la **Definición de Hecho (*Definition of Done - DoD*)**:

- Los requisitos no contienen adjetivos subjetivos ni funciones no verificables.
- El diagrama de bloques funcional está correctamente enlazado en Markdown mediante rutas relativas y se renderiza con nitidez en GitHub.
- La tabla de recursos distingue con claridad el equipamiento del cliente de las herramientas de desarrollo.
- El archivo `sprint2-backlog.md` refleja la realidad del repositorio: no pueden quedar casillas vacías (`- [ ]`) si el incremento se da por entregado.

C. TÉCNICAS DE ORATORIA TÉCNICA Y DEFENSA DEL INCREMENTO V0.2

Defender un catálogo de requisitos y un modelo de arquitectura ante un cliente o un comité evaluador requiere una técnica de comunicación específica:

- **No leer el documento.** La audiencia ya puede leer la pantalla. El ponente debe explicar el *porqué* de las decisiones de diseño.
- **Estructura de impacto en 3 minutos (*The 3-Minute Demo v0.2*):**
- *Minuto 1 (Requisitos nucleares).* Explicar los dos requisitos funcionales más críticos del proyecto y la principal restricción no funcional (ej. compatibilidad con OpenJDK 21 LTS y Maven).
- *Minuto 2 (Arquitectura en bloques).* Proyectar el diagrama de Draw.io y seguir el recorrido del dato desde el bloque de captura hasta la salida formateada.
- *Minuto 3 (Viabilidad y apertura).* Resumir los recursos asignados, certificar la viabilidad y anticipar el objetivo del Sprint 3 (planificación temporal mediante Diagrama de Gantt y matriz de riesgos).
- **Control de la comunicación no verbal y autoconfianza:** Mantener contacto visual con la sala, voz firme y pausada, postura erguida y dominio del vocabulario técnico sin vacilaciones.

Laboratorio. Cierre de la memoria y defensa técnica

Laia Claramunt asigna las tareas de la sesión:

*«Durante los próximos treinta minutos, cada uno de vosotros ensamblará el **Capítulo 2 consolidado de la Memoria del Proyecto (`capitulo2-requisitos-arquitectura.md`)** de su proyecto elegido. Comprobaréis que el diagrama de Draw.io se muestra impecable, cerraréis el **Sprint Backlog 2 al 100 % de cumplimiento** y redactaréis el guion de vuestra presentación oral de tres minutos».*

PASO 1. REDACCIÓN DEL CAPÍTULO 2 (`pi/docs/capitulo2-requisitos-arquitectura.md`)

1. Abre tu proyecto en **IntelliJ IDEA**.
2. Ensambla y unifica los contenidos trabajados en las semanas 4 y 5 en el archivo `memoria.md` de AzaharTech para **tu proyecto de la bolsa de proyectos**:

PASO 2. CIERRE Y VERIFICACIÓN DEL SPRINT BACKLOG 2 (`pi/backlog/sprint2-backlog.md`)

1. Abre tu archivo `sprint2-backlog.md`.
2. Actualiza **todas las tareas del sprint marcándolas como completadas (`- [x]`)**:

```
# Sprint Backlog 2 — [nombre de tu proyecto]

## Tareas

### Módulo: Proyecto Intermodular (PI)
- [x] T-PI-06: Redactar el catálogo de requisitos funcionales y no funcionales (`pi/docs/requisitos-sistema.md`)
- [x] T-PI-07: Contrastar fuentes técnicas oficiales y registrar el *Prompt Log*
- [x] T-PI-08: Modelar el diagrama de bloques funcional en Draw.io (`pi/docs/img/diagrama-bloques.png`)
- [x] T-PI-09: Consolidar el Capítulo 2 de la memoria y preparar el guion de defensa técnica v0.2

### Módulo: Entornos de Desarrollo (ED)
- [x] T-ED-05: Comprender el ciclo de compilación a Bytecode y la virtualización en la JVM
- [x] T-ED-06: Crear y configurar la estructura de proyecto con Apache Maven (`pom.xml`)
- [x] T-ED-07: Configurar y verificar el archivo `.gitignore` para exclusiones de compilación de Maven (`target/`)

### Módulo: Programación (PR)
- [x] T-PR-04: Instanciar y utilizar objetos de clases predefinidas (`Scanner`, `String`, `Math`, `Random`)
- [x] T-PR-05: Programar llamadas a métodos estáticos y métodos de instancia pasando parámetros
- [x] T-PR-06: Integrar los cálculos y generación de cadenas en el código maestro del proyecto propio
```

PASO 3. PREPARACIÓN DE LA FICHA DE GUION DE EXPOSICIÓN (3 MINUTOS)

Añade al final de tu documento consolidado o en tu libreta técnica el guion para la *Sprint Review*:

```
### Guion de Exposición Técnica: Sprint Review v0.2 (3 Minutos)
* **00:00 - 01:00 (Catálogo de Requisitos).** Saludo formal, nombre del proyecto, cliente y explicación concisa de los dos requisitos funcionales principales (RF-01 y RF-02) y la restricción tecnológica (RNF-01: OpenJDK 21).
* **01:00 - 02:00 (Arquitectura Visual).** Proyección del diagrama de bloques de Draw.io, explicando el recorrido del dato desde el subsistema de entrada hasta la salida estructurada.
* **02:00 - 03:00 (Viabilidad y Apertura al Sprint 3).** Certificación de la viabilidad técnica y recursos materiales, confirmación del cierre de tareas en GitHub y anticipación del objetivo del Sprint 3 (planificación temporal mediante Diagrama de Gantt y gestión de riesgos).
```

PASO 4. CONFIRMACIÓN Y SINCRONIZACIÓN EN GITHUB

1. Abre el panel **Commit** en IntelliJ IDEA (`Ctrl + K`).
2. Comprueba que aparecen seleccionados todos los archivos modificados durante el dia de hoy.
3. Escribe el mensaje siguiendo el estándar de **Conventional Commits**:

```
docs(pi): consolidar memoria técnica final del sprint 2 y cerrar backlog al 100%
```

4. Haz clic en **Commit and Push** y comprueba que `memoria.md` se visualiza con su formato Markdown, la imagen de Draw.io y todas las casillas del backlog marcadas con el check `[x]` .

3.3.2 Resumen del Sprint 2 completado

Al concluir estas 3 sesiones de los viernes (3 horas lectivas en total):

1. Has formalizado el **catálogo oficial de requisitos del sistema**, clasificando con precisión los Requisitos Funcionales (RF) y No Funcionales (RNF) bajo criterios verificables y sin ambigüedades.
2. Has modelado visualmente la arquitectura técnica mediante **Draw.io**, exportando e incrustando el **Diagrama de Bloques Funcional (IPO)** con tratamiento profesional de imágenes técnicas en Markdown.
3. Has justificado con rigor la **previsión de recursos materiales y humanos**, contrastando fuentes oficiales de la industria (Oracle y Apache Maven) y manteniendo el registro ético de IA mediante el **Prompt Log del Sprint 2**.
4. Cuentas con el **Capítulo 2 de la Memoria Técnica consolidado**, el **Sprint Backlog 2 al 100 %** y un guion ensayado para la **defensa técnica del incremento v0.2**, dejando el proyecto plenamente preparado para abordar en el Sprint 3 la planificación temporal mediante diagramas de Gantt y la gestión de riesgos.